Complete upload script used by all four API clients (Quiz Shorts, Satisfying Sims, BrainSprint, Data Race Uploader)

Owner: Navjeet Singh Malli (navjeetmalli@gmail.com). Each channel runs this same code with its own Google Cloud project, OAuth client and refresh token. A macOS launchd job runs run-daily.sh at fixed times; it renders one original video and uploads it to the owner's own channel.

run-daily.sh (scheduler entry point, BrainSprint channel; the other 3 channels use the same pattern)

```
#!/bin/bash
# run-daily.sh — render one BrainSprint Short (kinds rotate) and upload it if the channel token exists.
set -u
export PATH="/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin:$PATH"   # launchd gives a bare PATH (no ffmpeg)
cd "$(dirname "$0")"; source ./config.sh
mkdir -p logs state out
exec >> logs/run.log 2>&1
# CATCH-UP guard: RunAtLoad fires at every boot; skip if the last run finished < ~2h ago.
last=$(grep "run end" logs/run.log 2>/dev/null | tail -1 | awk '{print $2" "$3}')
if [ -n "$last" ]; then
  age=$(( $(date +%s) - $(date -j -f "%Y-%m-%d %H:%M:%S" "$last" +%s 2>/dev/null || echo 0) ))
  [ "$age" -lt 7000 ] && { echo "$(date '+%F %T') skip: last run ${age}s ago"; exit 0; }
fi
echo "=== $(date '+%F %T') run start"
if security find-generic-password -a "$USER" -s "$KC_REFRESH" -w >/dev/null 2>&1 && [ -f "$CLIENT_SECRET_FILE" ]; then
  UPLOAD=1; else echo "no token/client secret for $KC_REFRESH yet (generating only)"; UPLOAD=0; fi
n=$(cat state/run_counter 2>/dev/null || echo 0); set -- $KINDS; mode=$(eval echo \${$(( n % $# + 1 ))}); echo $((n+1))
> state/run_counter
if mp4=$("$PY" make_video.py --kind "$mode" | tail -1); then
  echo "generated $mp4"
  [ "$UPLOAD" = 1 ] && { "$PY" upload.py "${mp4%.mp4}.json" --privacy "$PRIVACY" --playlist "$PLAYLIST_TITLE" --
category 27 || echo "upload failed ($mode)"; }
  [ "$UPLOAD" = 1 ] && { "$PY" social_post.py "${mp4%.mp4}.json" 2>&1 | tail -3 || echo "social post failed ($mode)"; }
# Facebook + Instagram Reels
else echo "generate failed ($mode)"; fi
find out -name "*.mp4" -mtime +14 -delete   # keep two weeks of renders
echo "=== $(date '+%F %T') run end"
```

upload.py (uploads one video + thumbnail + playlist entry via YouTube Data API v3)

```
#!/usr/bin/env python
"""upload.py — upload a generated video (+thumbnail, +playlist) to YouTube.

Usage: upload.py <meta.json> [--privacy public|unlisted|private] [--playlist "Title"] [--dry-run]
Auth: reuses ~/sgpc-youtube-live/youtube_api.py (OAuth client + Keychain refresh token).
Set env KC_REFRESH=<keychain service> to upload to a different channel's token
(e.g. the NEW channel: run oauth-setup.sh in this folder first, see README).
"""
import argparse, json, os, re, sys, urllib.request, urllib.error
sys.path.insert(0, os.path.expanduser("~/sgpc-youtube-live"))
import youtube_api as y
if os.environ.get("KC_REFRESH"): y.KC_REFRESH = os.environ["KC_REFRESH"]
if os.environ.get("CLIENT_SECRET_FILE"): y.CLIENT_SECRET_FILE = os.environ["CLIENT_SECRET_FILE"]

UPLOAD = "https://www.googleapis.com/upload/youtube/v3"

def api(method, path, tok, body=None, params=""):
    return y._http(method, f"{y.YT_BASE}/{path}?{params}", headers={"Authorization": "Bearer " + tok}, body_json=body)

def resumable_upload(tok, meta, privacy, category="22"):
    body = {"snippet": {"title": meta["title"][:100], "description": meta["description"][:4900],
                        "tags": meta.get("tags", [])[:30], "categoryId": category, "defaultLanguage": "en"},
            "status": {"privacyStatus": privacy, "selfDeclaredMadeForKids": False}}
    size = os.path.getsize(meta["video"])
    req = urllib.request.Request(f"{UPLOAD}/videos?uploadType=resumable&part=snippet,status",
        data=json.dumps(body).encode(), method="POST",
        headers={"Authorization": "Bearer " + tok, "Content-Type": "application/json; charset=UTF-8",
                 "X-Upload-Content-Length": str(size), "X-Upload-Content-Type": "video/mp4"})
```

```
    try:
        with urllib.request.urlopen(req, timeout=60) as r: loc = r.headers["Location"]
    except urllib.error.HTTPError as e:
        raise y.ApiError(e.code, e.read().decode("utf-8", "replace"))
    with open(meta["video"], "rb") as f: data = f.read()
    req = urllib.request.Request(loc, data=data, method="PUT",
        headers={"Authorization": "Bearer " + tok, "Content-Type": "video/mp4", "Content-Length": str(size)})
    try:
        with urllib.request.urlopen(req, timeout=1800) as r: return json.loads(r.read().decode())
    except urllib.error.HTTPError as e:
        raise y.ApiError(e.code, e.read().decode("utf-8", "replace"))

def set_thumbnail(tok, vid, path):
    with open(path, "rb") as f: data = f.read()
    req = urllib.request.Request(f"{UPLOAD}/thumbnails/set?videoId={vid}", data=data, method="POST",
        headers={"Authorization": "Bearer " + tok, "Content-Type": "image/png", "Content-Length": str(len(data))})
    try:
        with urllib.request.urlopen(req, timeout=120) as r: return json.loads(r.read().decode())
    except urllib.error.HTTPError as e:
        print("thumbnail failed (non-fatal):", e.code, e.read().decode()[:200])

def ensure_playlist(tok, title, state_file):
    if os.path.exists(state_file): return open(state_file).read().strip()
    r = api("POST", "playlists", tok, {"snippet": {"title": title}, "status": {"privacyStatus": "public"}},
"part=snippet,status")
    os.makedirs(os.path.dirname(state_file), exist_ok=True); open(state_file, "w").write(r["id"]); return r["id"]

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("meta"); ap.add_argument("--privacy", default="public")
    ap.add_argument("--playlist", action="append"); ap.add_argument("--dry-run", action="store_true")
    ap.add_argument("--category", default="22", help="YouTube categoryId (22 People & Blogs, 27 Education)")
    a = ap.parse_args()
    meta = json.load(open(a.meta))
    done = a.meta + ".uploaded"
    if os.path.exists(done): print("already uploaded:", open(done).read().strip()); return
    tok = y.get_access_token()
    ch = api("GET", "channels", tok, params="part=snippet&mine=true")["items"][0]
    print(f"channel: {ch['snippet']['title']} ({ch['id']})  ->  {meta['title']}")
    if a.dry_run: print("dry-run: not uploading"); return
    r = resumable_upload(tok, meta, a.privacy, a.category)
    vid = r["id"]; st = r.get("status", {})
    print("uploaded:", f"https://youtu.be/{vid}", "| uploadStatus:", st.get("uploadStatus"), "| privacy:",
st.get("privacyStatus"))
    if st.get("privacyStatus") != a.privacy:
        print("NOTE: YouTube forced privacy to", st.get("privacyStatus"), "— API project probably needs the audit (see
README).")
    if meta.get("thumbnail") and os.path.exists(meta["thumbnail"]): set_thumbnail(tok, vid, meta["thumbnail"])
    titles = list(a.playlist or [])
    proj = os.path.abspath(os.path.join(os.path.dirname(os.path.abspath(a.meta)), ".."))
    if os.path.exists(os.path.join(proj, "playlists.py")):   # category + format playlists, only for the project that
owns the video
        sys.path.insert(0, proj)                             # (upload.py is symlinked into other projects: don't pick
up quiz's rules there)
        import playlists; titles += [t for t in playlists.titles_for(meta["id"]) if t not in titles]
    for t in titles:
        slug = re.sub(r"[^a-z0-9]+", "-", t.lower()).strip("-")   # one state file per playlist title
        try:
            pid = ensure_playlist(tok, t, os.path.join(os.path.dirname(os.path.abspath(a.meta)), "..", "state",
f"playlist_{slug}"))
            api("POST", "playlistItems", tok, {"snippet": {"playlistId": pid, "resourceId": {"kind": "youtube#video",
"videoId": vid}}}, "part=snippet")
        except Exception as e: print("playlist failed (non-fatal):", t, str(e)[:120])
    open(done, "w").write(vid + "\n")

if __name__ == "__main__":
    main()
```

youtube_api.py (OAuth 2.0 token handling; secrets are read from client_secret.json and the macOS Keychain, never stored in code)

```
"""
youtube_api.py — small wrapper for the YouTube Data API v3.
```

```
Uses only the Python standard library (no pip install needed).
Reads OAuth credentials from `client_secret.json` and the refresh token from
the macOS Keychain.
"""
from __future__ import annotations   # PEP 563: keep annotations as strings (3.9-safe)

import json
import os
import subprocess
import sys
import urllib.error
import urllib.parse
import urllib.request

DIR = os.path.dirname(os.path.abspath(__file__))
CLIENT_SECRET_FILE = os.path.join(DIR, "client_secret.json")
STATE_DIR = os.path.join(DIR, "state")
AUTH_BROKEN_FILE = os.path.join(STATE_DIR, "auth-broken")   # sentinel for status.sh / supervisor

# Keychain entries — must match config.sh.
KC_REFRESH = "sgpc-youtube-oauth-refresh"     # refresh_token
KC_STREAM_ID = "sgpc-youtube-stream-id"       # YouTube liveStream resource id
KC_STREAM_KEY = "sgpc-youtube-stream-key"     # ingestion key (rtmp)

OAUTH_TOKEN_URL = "https://oauth2.googleapis.com/token"
YT_BASE = "https://www.googleapis.com/youtube/v3"
# youtube.force-ssl is required to write liveBroadcasts/liveStreams.
SCOPES = "https://www.googleapis.com/auth/youtube.force-ssl"


class ApiError(Exception):
    """Carries the parsed error body so callers can branch on error codes."""
    def __init__(self, code: int, body: str):
        super().__init__(f"HTTP {code}: {body}")
        self.code = code
        self.body = body
        try:
            self.data = json.loads(body)
        except Exception:
            self.data = {}


# ---------- Keychain (macOS) helpers ----------------------------------------
def keychain_get(service: str):
    try:
        out = subprocess.run(
            ["security", "find-generic-password", "-a", os.environ["USER"],
             "-s", service, "-w"],
            check=True, capture_output=True, text=True,
        )
        return out.stdout.strip() or None
    except subprocess.CalledProcessError:
        return None


def keychain_set(service: str, value: str) -> None:
    # NOTE: the secret is passed on argv. macOS `security` does not read the
    # password from stdin for add-generic-password (it forces interactive
    # double-entry), so argv is the only non-interactive option. Exposure is
    # minimal: this runs only during one-time setup, briefly, and any process
    # that could read it via `ps` already has Keychain read access as this user.
    subprocess.run(
        ["security", "add-generic-password", "-U", "-a", os.environ["USER"],
         "-s", service, "-w", value],
        check=True, capture_output=True, text=True,
    )


# ---------- auth-broken sentinel --------------------------------------------
def _set_auth_broken(reason: str) -> None:
    try:
        os.makedirs(STATE_DIR, exist_ok=True)
        with open(AUTH_BROKEN_FILE, "w") as f:
            f.write(reason + "\n")
    except OSError:
        pass
```

```
def _clear_auth_broken() -> None:
    try:
        os.remove(AUTH_BROKEN_FILE)
    except OSError:
        pass


# ---------- OAuth client info ------------------------------------------
def load_client_secret():
    if not os.path.isfile(CLIENT_SECRET_FILE):
        sys.exit(
            f"ERROR: {CLIENT_SECRET_FILE} not found.\n"
            "       Follow OAUTH_SETUP.md to create a Google Cloud OAuth Client ID\n"
            "       (Desktop app) and save the downloaded JSON to that path."
        )
    # Tighten permissions if the download left it world/group-readable.
    try:
        if os.stat(CLIENT_SECRET_FILE).st_mode & 0o077:
            os.chmod(CLIENT_SECRET_FILE, 0o600)
    except OSError:
        pass

    with open(CLIENT_SECRET_FILE) as f:
        data = json.load(f)

    info = data.get("installed")
    if info is None:
        if "web" in data:
            sys.exit(
                "ERROR: client_secret.json is a 'Web application' OAuth client.\n"
                "       This tool needs a 'Desktop app' client (it uses a loopback\n"
                "       redirect that only Desktop clients accept). Redo Step 4 of\n"
                "       OAUTH_SETUP.md and choose 'Desktop app'."
            )
        sys.exit("ERROR: client_secret.json has no 'installed' section — not a Desktop OAuth client.")

    cid = info.get("client_id")
    csec = info.get("client_secret")
    if not cid or not csec:
        sys.exit("ERROR: client_secret.json is missing client_id / client_secret.")
    return cid, csec


# ---------- Low-level HTTP ---------------------------------------------
def _http(method: str, url: str, *, headers=None, data=None, body_json=None):
    if body_json is not None:
        data = json.dumps(body_json).encode("utf-8")
        headers = {**(headers or {}), "Content-Type": "application/json"}
    req = urllib.request.Request(url, data=data, headers=headers or {}, method=method)
    try:
        with urllib.request.urlopen(req, timeout=30) as resp:
            return json.loads(resp.read().decode("utf-8") or "{}")
    except urllib.error.HTTPError as e:
        body = e.read().decode("utf-8", "replace")
        raise ApiError(e.code, body)


def get_access_token() -> str:
    """Exchange the stored refresh_token for a fresh access_token."""
    refresh = keychain_get(KC_REFRESH)
    if not refresh:
        sys.exit("ERROR: no OAuth refresh_token. Run ./oauth-setup.sh first.")
    cid, csec = load_client_secret()
    body = urllib.parse.urlencode({
        "client_id": cid, "client_secret": csec,
        "refresh_token": refresh, "grant_type": "refresh_token",
    }).encode()
    try:
        resp = _http("POST", OAUTH_TOKEN_URL, data=body,
                     headers={"Content-Type": "application/x-www-form-urlencoded"})
    except ApiError as e:
        err = (e.data or {}).get("error", "")
        # All permanent token-grant failures that require re-running oauth-setup.
        # (Transient/unknown errors fall through to raise so we don't false-alarm.)
```

```
        if err in ("invalid_grant", "invalid_client", "unauthorized_client",
                   "unsupported_grant_type", "deleted_client"):
            _set_auth_broken(err)
            sys.exit(
                f"AUTH BROKEN: refresh token rejected ({err}). Re-run ./oauth-setup.sh.\n"
                "       Common cause: the OAuth consent screen is still in 'Testing' mode,\n"
                "       which expires refresh tokens after 7 days. Publish the app (see\n"
                "       OAUTH_SETUP.md) to stop this recurring."
            )
        print(f"Token refresh failed: {e}", file=sys.stderr)
        raise
    tok = resp.get("access_token")
    if not tok:
        sys.exit(f"ERROR: token refresh returned no access_token. Keys: {sorted(resp.keys())}")
    _clear_auth_broken()   # we got a token -> auth is healthy again
    return tok


def api(method: str, path: str, *, params=None, body=None) -> dict:
    """Authenticated JSON call to /youtube/v3/<path>."""
    qs = ("?" + urllib.parse.urlencode(params)) if params else ""
    return _http(method, f"{YT_BASE}/{path}{qs}",
                 headers={"Authorization": f"Bearer {get_access_token()}"},
                 body_json=body)
```

oauth-browser.py (one-time OAuth consent for the channel owner, loopback redirect)

```
#!/usr/bin/env python3
"""oauth-browser.py — OAuth for the NEW channel without a terminal.
  start            : prints the Google auth URL, saves PKCE state, waits (10 min) for the loopback redirect,
                     exchanges the code and stores the refresh token in Keychain under KC_REFRESH.
  finish <url>     : if the loopback redirect could not be caught, pass the final browser URL (contains ?code=...).
"""
import base64, hashlib, json, os, secrets, sys, threading, time, urllib.parse, urllib.request, http.server
sys.path.insert(0, os.path.expanduser("~/sgpc-youtube-live"))
import youtube_api as _y
if os.environ.get("CLIENT_SECRET_FILE"): _y.CLIENT_SECRET_FILE = os.environ["CLIENT_SECRET_FILE"]   # per-channel
Google Cloud project
from youtube_api import OAUTH_TOKEN_URL, SCOPES, keychain_set, load_client_secret
KC = os.environ.get("KC_REFRESH", "quiz-youtube-oauth-refresh")
SCOPES = os.environ.get("OAUTH_SCOPES", SCOPES)
DIR = os.path.dirname(os.path.abspath(__file__)); ST = os.path.join(DIR, "state"); os.makedirs(ST, exist_ok=True)
PENDING = os.path.join(ST, "oauth_pending.json"); RESULT = os.path.join(ST, "oauth_result.txt")
PORT = 8765

def exchange(code, p):
    cid, csec = load_client_secret()
    body = urllib.parse.urlencode({"code": code, "client_id": cid, "client_secret": csec, "redirect_uri":
p["redirect_uri"],
                                   "grant_type": "authorization_code", "code_verifier": p["verifier"]}).encode()
    req = urllib.request.Request(OAUTH_TOKEN_URL, data=body, headers={"Content-Type": "application/x-www-form-
urlencoded"}, method="POST")
    try:
        with urllib.request.urlopen(req, timeout=30) as r: tok = json.loads(r.read().decode())
    except urllib.error.HTTPError as e:
        open(RESULT, "w").write("ERROR exchange: " + e.read().decode()[:300]); raise
    refresh = tok.get("refresh_token")
    if not refresh: open(RESULT, "w").write("ERROR no refresh_token"); sys.exit("no refresh_token")
    keychain_set(KC, refresh); open(RESULT, "w").write("OK stored in " + KC); os.remove(PENDING); print("OK stored in",
KC)

class H(http.server.BaseHTTPRequestHandler):
    code = None
    def log_message(self, *a): pass
    def do_GET(self):
        q = urllib.parse.parse_qs(urllib.parse.urlparse(self.path).query)
        p = json.load(open(PENDING))
        if (q.get("state") or [None])[0] != p["state"]: self.send_response(400); self.end_headers(); return
        H.code = (q.get("code") or [None])[0]
        self.send_response(200); self.send_header("Content-Type", "text/html"); self.end_headers()
        self.wfile.write(b"<h2>Authorization received. You can close this tab.</h2>")

def start():
    cid, _ = load_client_secret()
```

```
    verifier = secrets.token_urlsafe(64)
    challenge = base64.urlsafe_b64encode(hashlib.sha256(verifier.encode()).digest()).rstrip(b"=").decode()
    p = {"state": secrets.token_urlsafe(16), "verifier": verifier, "redirect_uri": f"http://127.0.0.1:{PORT}/callback"}
    json.dump(p, open(PENDING, "w"))
    url = "https://accounts.google.com/o/oauth2/v2/auth?" + urllib.parse.urlencode({
        "client_id": cid, "redirect_uri": p["redirect_uri"], "response_type": "code", "scope": SCOPES,
        "access_type": "offline", "prompt": "consent", "state": p["state"], "code_challenge": challenge,
"code_challenge_method": "S256"})
    open(os.path.join(ST, "oauth_url.txt"), "w").write(url); print(url, flush=True)
    if os.path.exists(RESULT): os.remove(RESULT)
    srv = http.server.HTTPServer(("127.0.0.1", PORT), H); threading.Thread(target=srv.serve_forever,
daemon=True).start()
    t0 = time.time()
    while time.time() - t0 < 600 and H.code is None: time.sleep(0.3)
    srv.shutdown()
    if H.code: exchange(H.code, p)
    else: print("timed out waiting for redirect (use: finish <url>)")

def finish(url):
    p = json.load(open(PENDING)); q = urllib.parse.parse_qs(urllib.parse.urlparse(url).query)
    if (q.get("state") or [None])[0] != p["state"]: sys.exit("state mismatch")
    exchange(q["code"][0], p)

if __name__ == "__main__":
    {"start": start, "finish": lambda: finish(sys.argv[2])}[sys.argv[1]]()
```